



13

LASER AUDIO DISPLAY WITH A RASPBERRY PI



In [Chapter 12](#), you used a Pico, a tiny microcontroller, to generate musical tones. In this chapter, you'll use a much more powerful embedded system, the Raspberry Pi, to produce interesting laser patterns based on audio signals.

The [previous chapter's](#) Pico had an RP2040 microcontroller with dual ARM Cortex-M0 processors running at speeds of up to 133 MHz, with 264KB of random access memory (RAM) and 2MB of nonvolatile storage on an external flash chip. The Raspberry Pi 3B+, in comparison, has a much more capable ARM Cortex-A53 processor operating at 1.4 GHz, with 1GB of RAM and storage of several gigabytes, depending on the SD card you use. While this still pales in comparison to a standard desktop or laptop computer, the Pi is nonetheless capable of running a Linux-based operating system and full-fledged Python, unlike the Pico.

In this chapter, you'll use Python on the Raspberry Pi to read an audio file in the WAV format, perform computations based on the real-time audio data, and use that data to adjust the speed and direction of rota-

tion of two motors in a laser display rig. You'll attach mirrors to the motors to reflect the beam from an inexpensive laser module, producing Spirograph-like patterns that change in response to the audio. You'll also simultaneously stream the audio to a speaker so you can hear the WAV file playing as you watch your laser light show.

This project will push your knowledge of Python further as you learn to use the Raspberry Pi. Here are some of the topics we'll cover:

- Generating interesting patterns with a laser and two rotating mirrors
- Getting frequency information from a signal using the fast Fourier transform (FFT)
- Computing FFTs using `numpy`
- Reading audio from WAV files
- Outputting audio data using `pyaudio`
- Driving motors with a Raspberry Pi
- Toggling a laser module on/off with a metal-oxide-semiconductor field-effect transistor (MOSFET)

How It Works

You'll use a Raspberry Pi to work with the audio data and control the hardware. **Figure 13-1** shows a block diagram of what you'll create in this project.

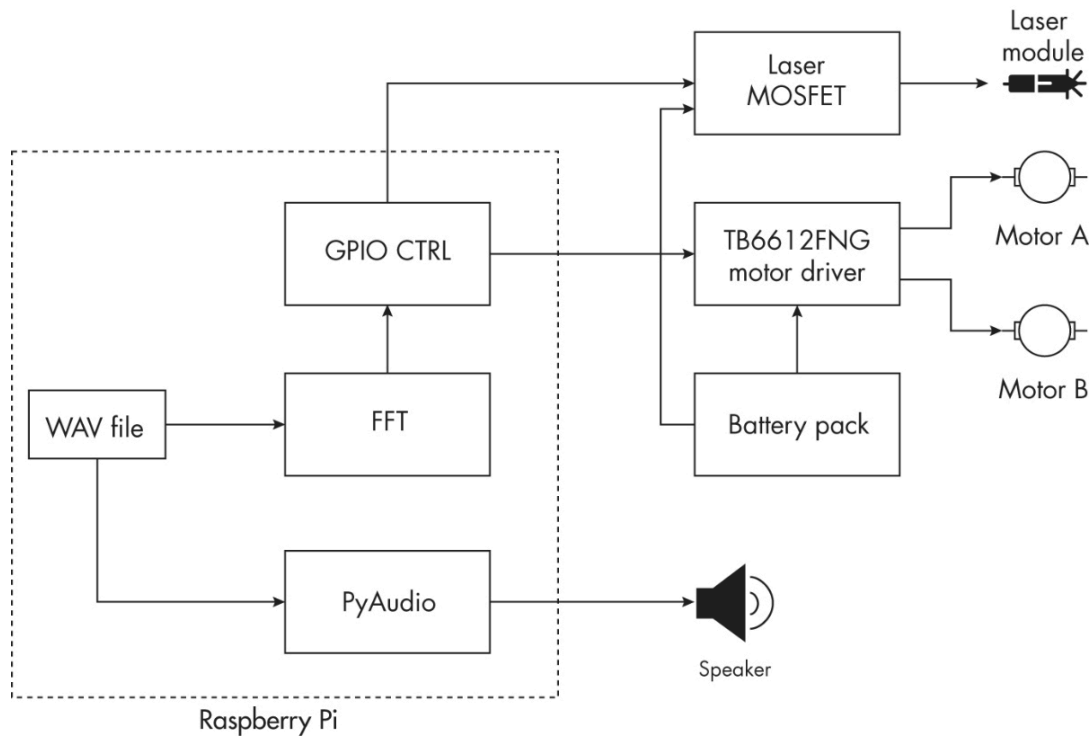


Figure 13-1: A block diagram of the laser audio project

The Raspberry Pi will use a WAV file in two ways. It will play the file through an attached speaker by way of `pyaudio`, while also analyzing the audio data in real time using a mathematical technique called a *fast Fourier transform (FFT)*. The Pi will use data from the FFT to drive the motors and laser via its general-purpose input/output (GPIO) pins, but to protect the Pi from damage, you won't hook it up directly to these external components. Instead, you'll connect it indirectly through a motor driver board and a MOSFET. Before you begin, let's consider in more detail how some of these aspects of the project will work.

Generating Patterns with a Laser

To generate the laser patterns in this project, you'll use a laser module and two mirrors attached to the shafts of two small DC motors, as shown in [Figure 13-2](#). Think of the laser as an intense beam of light that remains focused on a tiny point, even when projected over a large distance. This focus is possible because the beam is organized so that

its waves travel in one direction only and are in phase with each other. If you shine the laser at the surface of a flat mirror (mirror A in [Figure 13-2](#)), the reflection projected will remain a fixed point, even as the motor spins. Because the plane of reflection of the laser is perpendicular to the spinning axis of the motor, it's as if the mirror isn't rotating at all.

Now, say the mirror is attached at an angle relative to the motor shaft, as shown on the right of [Figure 13-2](#) (mirror B). As the shaft rotates, the projected point will trace an ellipse, and if the motor is spinning fast enough, the viewer will perceive the moving dot as a continuous shape.

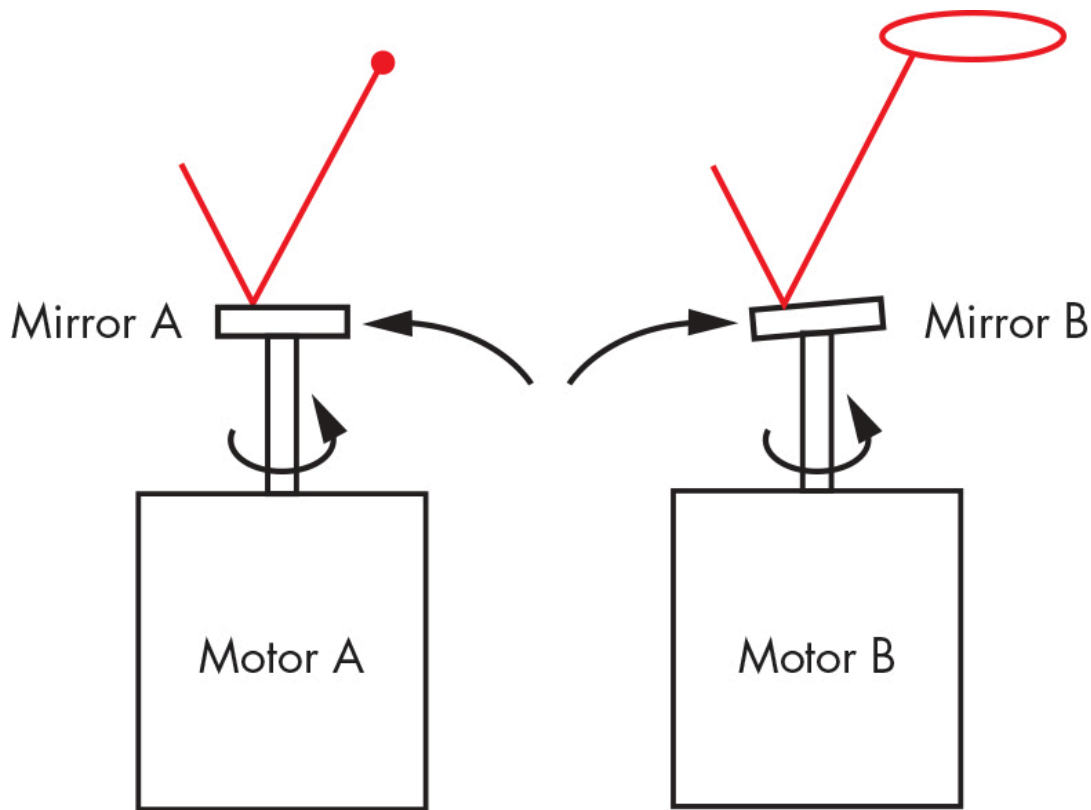


Figure 13-2: The flat mirror (mirror A) reflects a single dot. The reflection off the slanted mirror (mirror B) creates a circle as the motor spins.

What if both mirrors are slanted and you arrange them so that the point reflected off mirror A is projected onto mirror B? Now when motors A and B spin, the pattern created by the reflected point will be a

combination of the two rotational movements of motors A and B, producing interesting patterns, as shown in **Figure 13-3**.

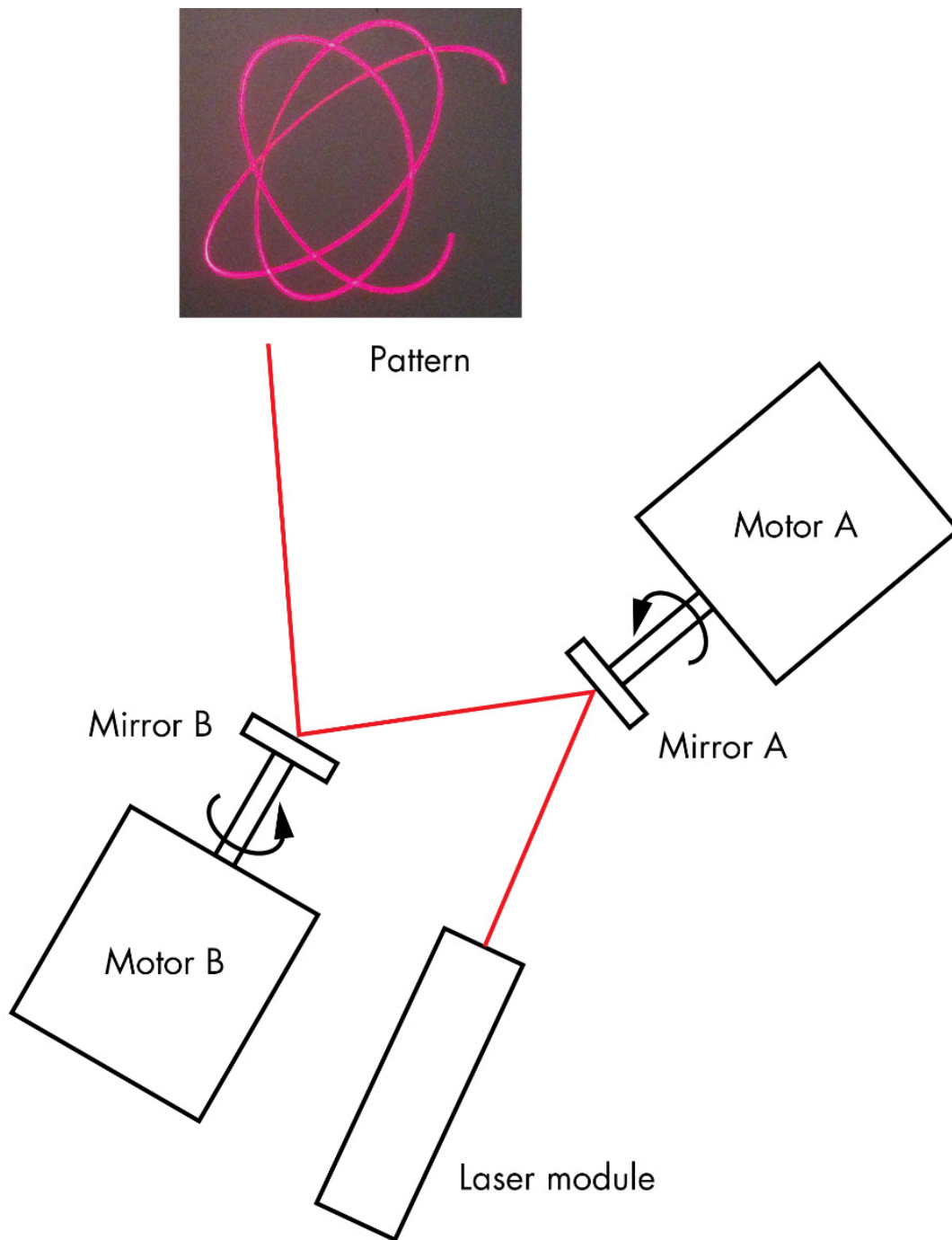


Figure 13-3: Reflecting laser light off two rotating, slanted mirrors produces interesting, complex patterns.

The exact patterns produced will depend on the speed and direction of rotation of the two motors, but they will be similar to the hypotrochoids produced by the Spirograph you explored in **Chapter 2**.

Motor Control

You'll use the Raspberry Pi to control the speed and direction of your motors through a technique called *pulse width modulation (PWM)*. This is a way to power a device (such as a motor) by sending digital pulses that switch on and off quickly, such that the device “sees” a continuous voltage. The signal sent to the device has a fixed frequency, but the fraction of time that the digital pulse is on, called the *duty cycle*, can vary. The duty cycle is expressed as a percentage. To illustrate, **Figure 13-4** shows three signals with the same frequency but different duty cycles—25 percent, 50 percent, and 75 percent.

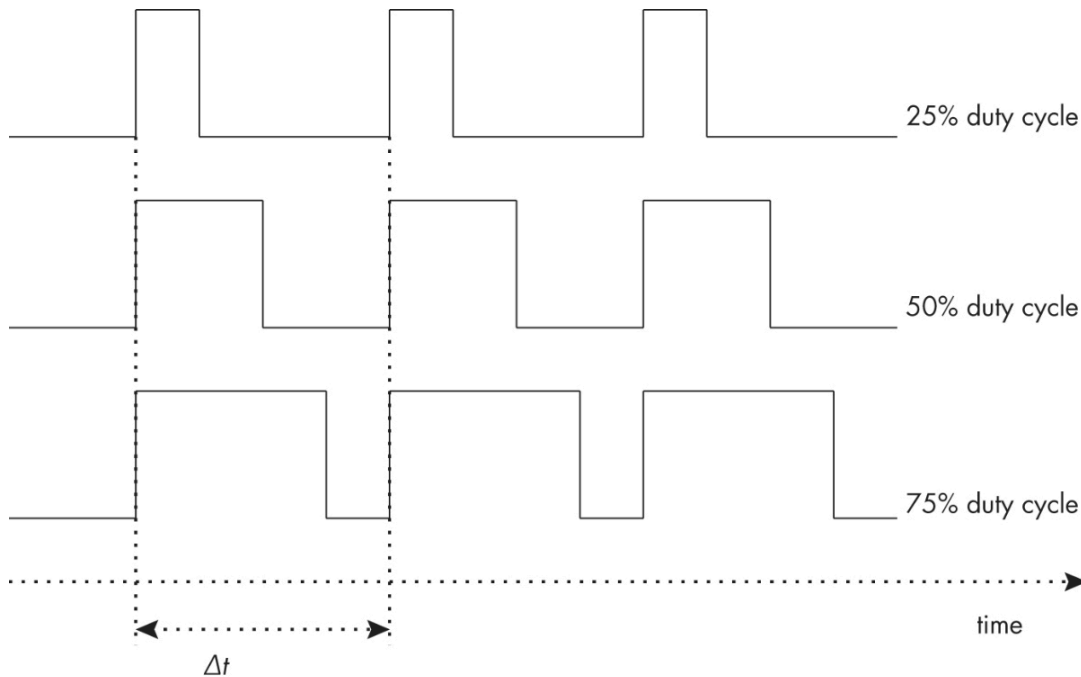


Figure 13-4: PWM signals with different duty cycles

The higher the duty cycle percentage, the longer the pulse is on for each cycle of the signal. The motor receiving the signal will perceive those longer pulses as a higher continuous voltage. By manipulating the duty cycle, you can thus provide varying power levels to the motors in this project, which will result in variations in motor speed and changes in the laser pattern.

NOTE *PWM has many applications beyond motor control. It could also be used, for example, to control the brightness of dimmable LEDs.*

Motors operate at a relatively high voltage, but the Raspberry Pi can handle only so much current before it's damaged. You'll be using a TB6612FNG motor driver breakout board, similar to the ones shown in [Figure 13-5](#), to act as an intermediary between the Pi and the motors, keeping the Pi safe. Many variants of this board are available, and you can choose any one of them, as long as you're careful to wire it up correctly.

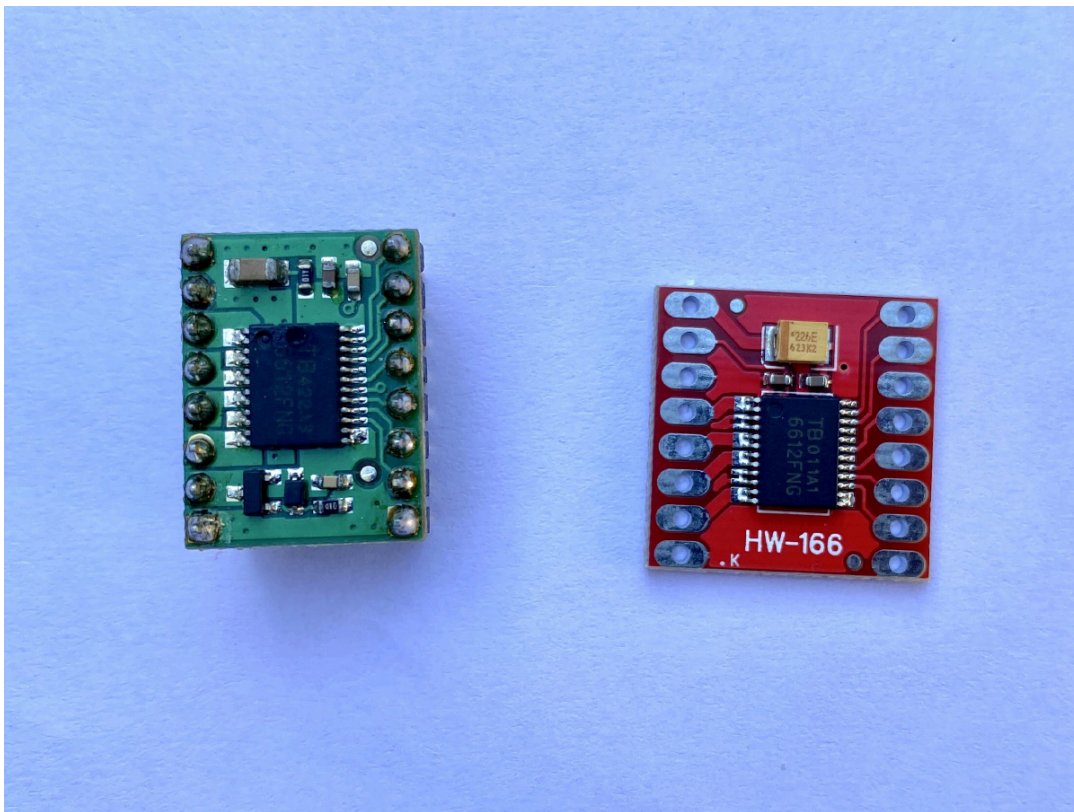


Figure 13-5: *TB6612FNG motor driver breakout printed circuit boards (PCBs)*

The bottom of the breakout board should have pin information. It's also a good idea to look at the TB6612FNG chip datasheet, which you can download from the internet. The *A* and *B* in the pin names denote the two motors. The *IN* pins control the direction of the motors, the *O1* and *O2* pins supply power to the motors, and the *PWM* pins use pulse width modulation to control the motor speeds. By writing to these

pins, you can control both the direction and speed of rotation for each motor, which is exactly what you need for this project. We won't go into the details of how this board works, but if you're curious, you can start by reading up on an *H-bridge*, a common circuit design that uses MOSFETs to control motors.

NOTE *You could replace this breakout part with any motor control circuit you're familiar with, as long as you modify the code appropriately.*

Laser Module

For the laser, you'll use an inexpensive laser module breakout PCB similar to the one shown in **Figure 13-6**.

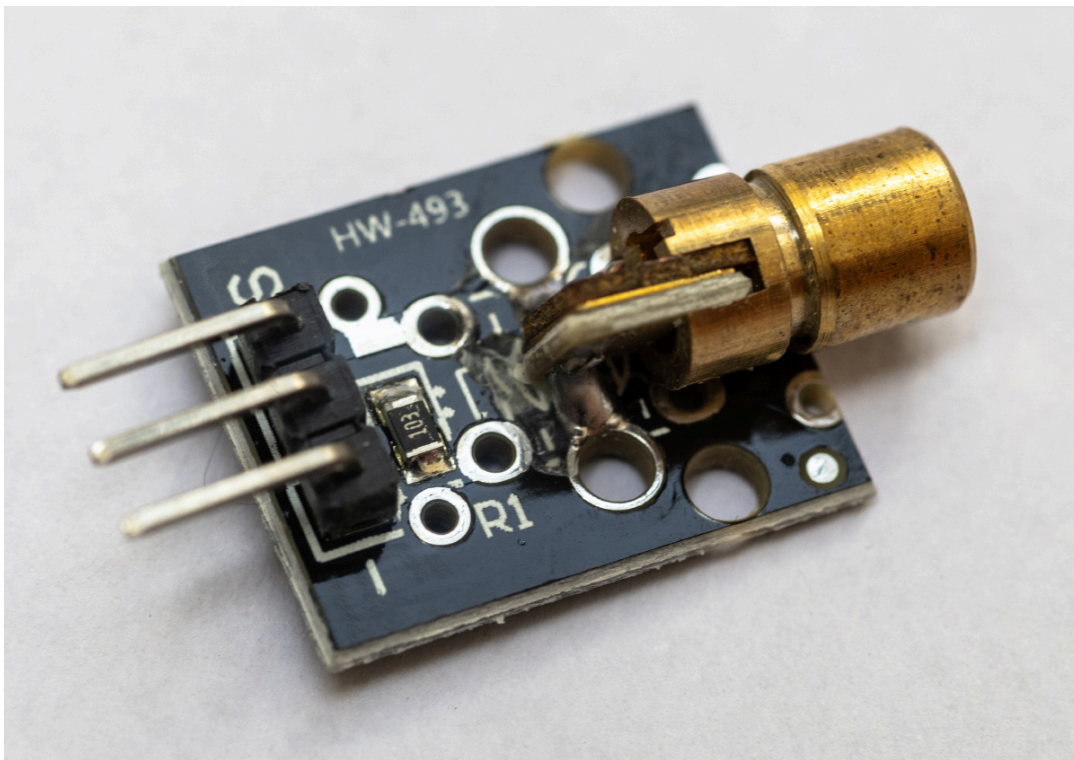


Figure 13-6: A laser module

Different variants of laser modules are available. You want one with a 650 nanometer (nm) red laser that operates at 5 volts (V). (The 650 nm refers to the laser's wavelength.) Be sure you understand the polarity

and connections of this board before you use it for the project. Test it separately using a 5 V power supply.

MOSFET

To turn the laser module on and off using the Raspberry Pi, you'll use an *N-channel MOSFET*, which you can think of as an electrically controlled switch. You can use almost any N-channel MOSFET for the project, but the BS170 is cheap and readily available. **Figure 13-7** shows the pin numbering for the MOSFET, as well as how to connect it to the laser module and the Raspberry Pi.

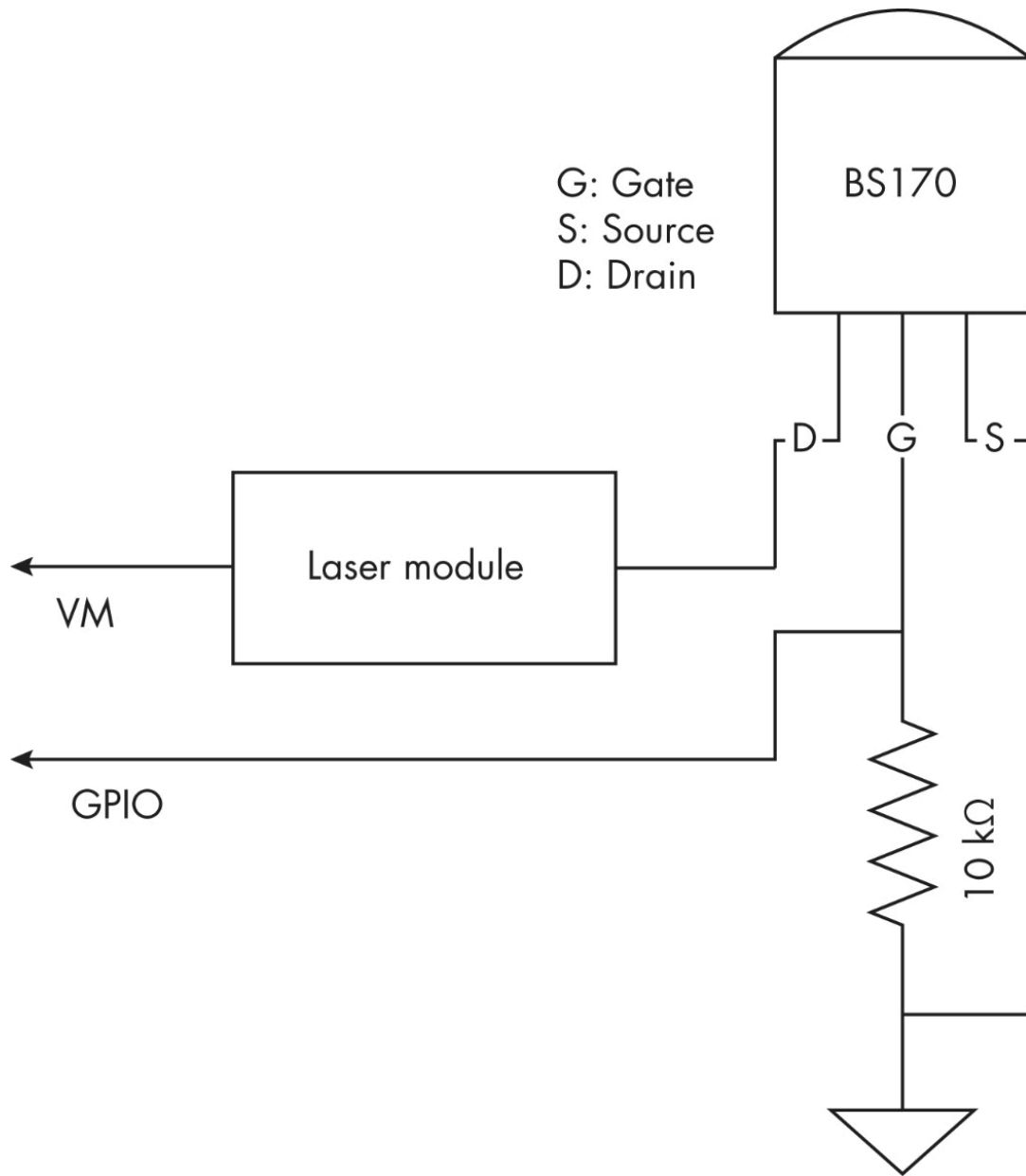


Figure 13-7: BS170 MOSFET connections

The 10 kΩ resistor “pulls” the gate pin of the MOSFET to the ground, so it isn’t triggered when the Raspberry Pi GPIO pin is in a floating state (for example, after a GPIO cleanup). When you send a HIGH to the GPIO, the MOSFET switch turns on, effectively connecting the laser module to VM and GND and powering it on.

Why do you need a MOSFET? Can’t you just connect the laser module directly to a GPIO pin of the Raspberry Pi? That’s not a great idea, because the MOSFET can take a lot more current than your Raspberry Pi.

Using a MOSFET isolates your Pi from a situation that causes a current spike on the load. Better to burn your cheap MOSFET than your comparatively expensive Raspberry Pi! In general, the MOSFET trick is a good thing to remember whenever you want to control an external device with your Raspberry Pi.

Analyzing Audio with the Fast Fourier Transform

Because the ultimate goal in this project is to control motor speeds based on audio input, you need to be able to analyze the audio in real time. Recall from **Chapter 4** that tones from an acoustic instrument are a mix of several frequencies, or overtones. In fact, any sound can be decomposed into its constituent frequencies using a *Fourier transform*. When the Fourier transform is applied to digital signals, the result is called the *discrete Fourier transform (DFT)* because digital signals are composed of many discrete samples. In this project, you'll use Python to implement a *fast Fourier transform (FFT)* algorithm to compute the DFT. (Throughout this chapter, I'll use *FFT* to refer to both the algorithm and the result.)

Figure 13-8 shows a simple example of an FFT. The top frame of the figure shows the waveform of a signal that combines just two sine waves. This plot is in the *time domain* because it's showing how the signal's amplitude varies over time. The bottom frame of the figure shows the FFT corresponding to that signal. The FFT is in the *frequency domain*; it's showing what frequencies are present in the signal at a given moment in time.

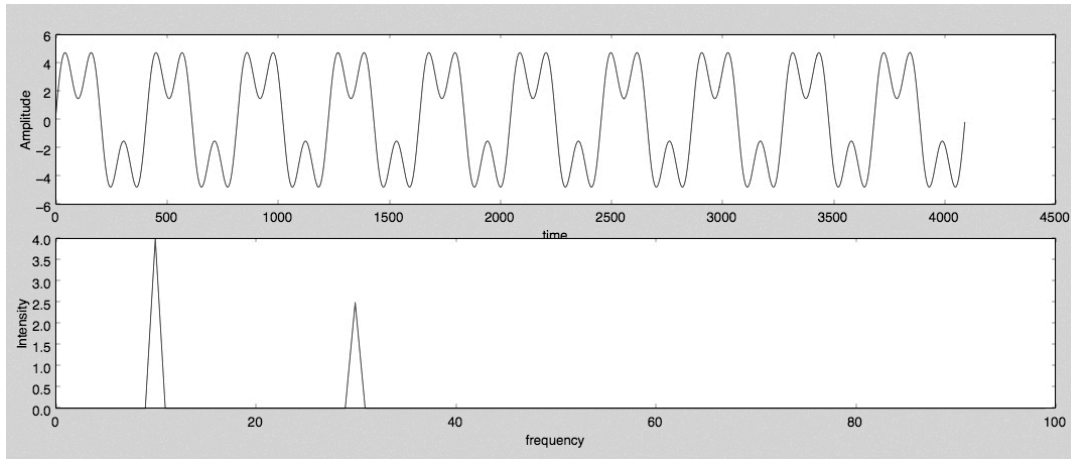


Figure 13-8: An audio signal containing multiple frequencies (top) and its corresponding FFT (bottom)

The wave in the top frame can be expressed by the following equation, which sums the two sine waves:

$$y(t) = 4\sin(2\pi 10t) + 2.5\sin(2\pi 30t)$$

Notice the 4 and 10 in the expression for the first wave—4 is the amplitude of the wave, and 10 is the frequency (in hertz). Meanwhile, the second wave has an amplitude of 2.5 and a frequency of 30 Hz. Take a look at the FFT in the bottom frame of the figure and you'll see it has two peaks, at 10 Hz and 30 Hz. The FFT has revealed the signal's component frequencies. The FFT also identifies the relative amplitude of each frequency; the intensity of the first peak is about twice that of the second peak.

Now let's look at a more realistic example. **Figure 13-9** shows a complex audio signal in the top frame and the corresponding FFT in the bottom frame. Notice that the FFT contains many more peaks at a variety of intensities, indicating that the signal contains many more frequencies.

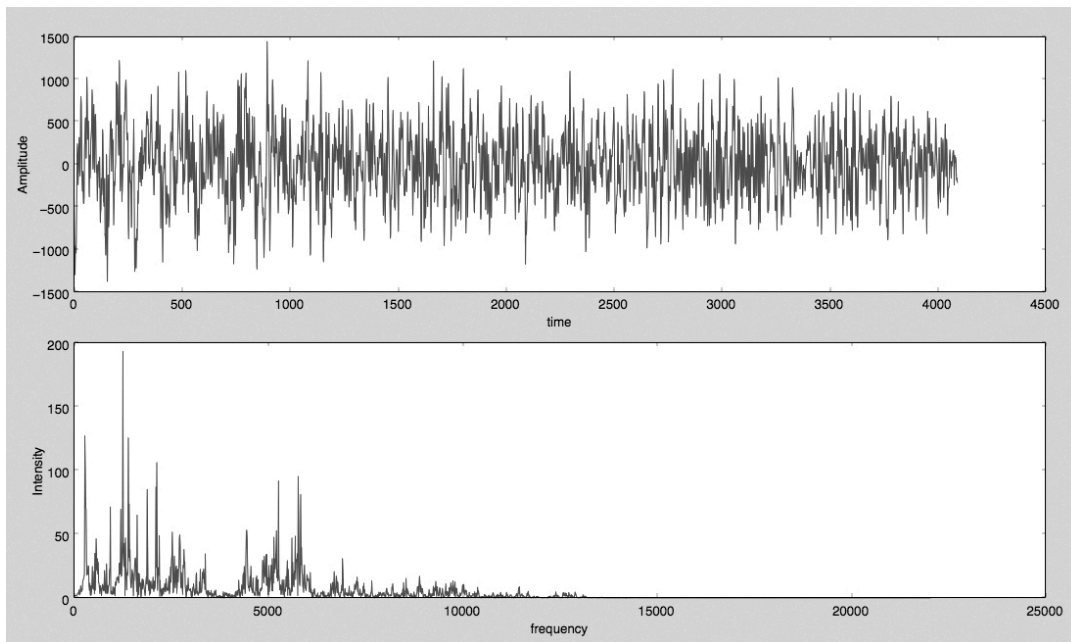


Figure 13-9: The FFT algorithm takes an amplitude signal (top) and computes its component frequencies (bottom).

To compute an FFT, you need a set of samples. The choice of the number of samples is a bit arbitrary, but a small sample size wouldn't give you a good picture of the signal's frequency content and might also mean a higher computational load because you would need to compute more FFTs per second. On the other hand, a sample size that's too large would average out the changes in the signal, so you wouldn't be getting a "real-time" frequency response for the signal. For the purposes of this project, a sample size of 2,048 will work. At a sampling rate of 44,100 Hz, 2,048 samples represent about 0.046 seconds of audio.

You'll use `numpy` to compute an FFT to split the audio data into its constituent frequencies, and then you'll use that information to control the motors. First you'll split the range of frequencies (in Hz) into three bands: `[0, 100]`, `[100, 1000]`, and `[1000, 2500]`. You'll compute an average amplitude level for each band, and each value will affect the motors and resulting laser pattern differently, as follows:

- Changes in the average amplitude of low frequencies will affect the speed of the first motor.

- Changes in the average amplitude of middle frequencies will affect the speed of the second motor.
- When high frequencies peak above a certain threshold, the first motor will change direction.

With these rules, the laser pattern will change in response to the audio signal.

Requirements

In this project, you'll use the following Python modules:

- `RPi.GPIO` to set up PWM and control the output of the pins
- `time` for pausing between operations
- `wave` to read WAV files
- `pyaudio` to process and stream audio data
- `numpy` for FFT computations
- `argparse` to process command line arguments

You'll also need the following items to build the project:

- One Raspberry Pi 3B+ or newer
- One 5 V adapter to power the Raspberry Pi
- One powered speaker with AUX (line-in) input (most Bluetooth speakers have an AUX input these days)
- One TB6612FNG motor breakout board
- One laser module breakout board
- One 10 k Ω resistor
- One BS170 N-channel MOSFET or equivalent
- Two DC motors like the ones used in a small toy, rated for 9 V
- Two small mirrors, approximately 1 inch or less in diameter
- One 3.7 V 18650 2000 mAh (3C) lithium-ion battery with holder (or use four AA batteries on a holder)
- Two 3D-printed parts to fix the mirrors onto the motor shafts (optional)

- One rectangular base, about 8 inches by 6 inches, to mount the hardware
- Some LEGO bricks to raise the motors and laser module off the base so the mirrors can spin freely
- A hot glue gun
- Superglue to fix the mirrors to the motor shafts
- A soldering iron
- A breadboard
- Wires to make connections (single-core hookup wires with male pins on both sides work nicely)

Setting Up the Raspberry Pi

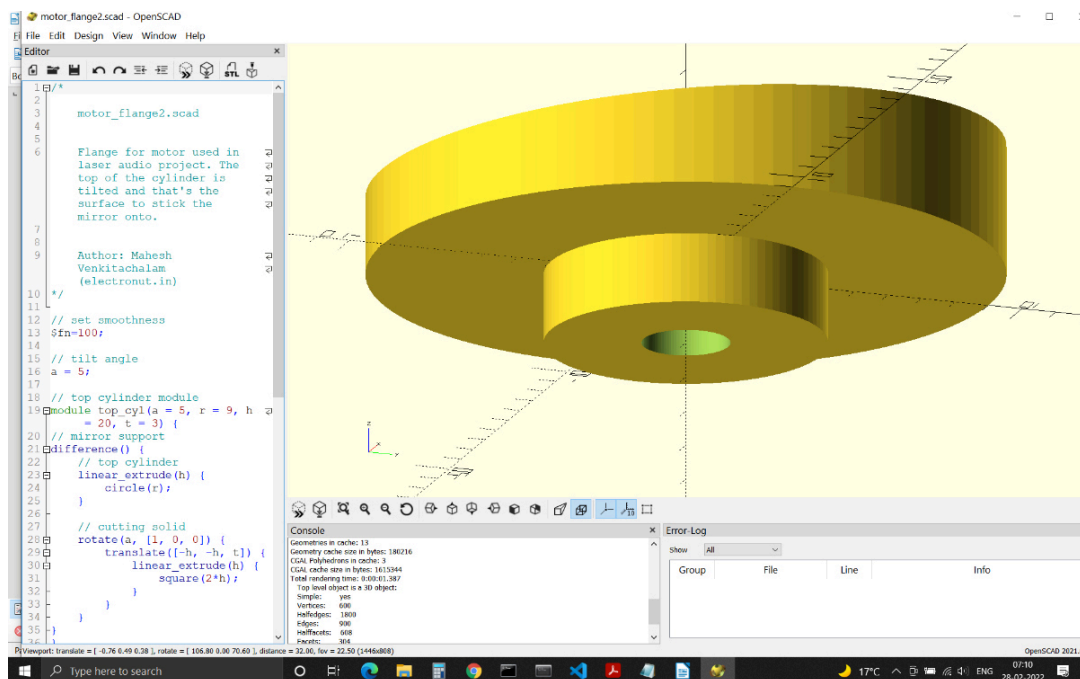
To set up your Raspberry Pi, see [Appendix B](#). Follow the instructions in the appendix, and make sure you've installed the `numpy` and `pyaudio` Python packages required for this project. You'll write code on the Raspberry Pi via a Secure Shell (SSH). You can set up Microsoft Visual Studio Code to work remotely on the Pi using an SSH from your laptop or desktop computer. This is also explained in [Appendix B](#).

Constructing the Laser Display

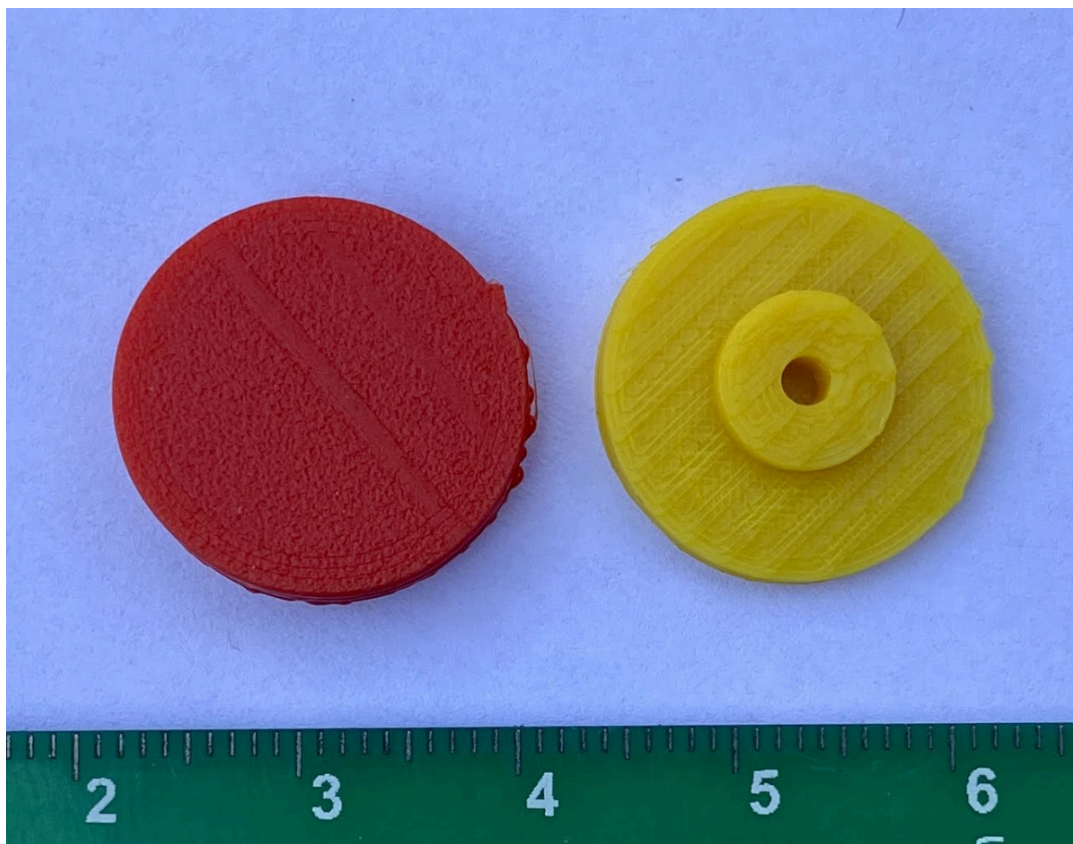
Before you hook up all the hardware, you should prepare the motors and the laser module for the laser display. The first order of business is to attach the mirrors to the motors. Each mirror has to be at a slight angle relative to the motor shaft. One way to do this would be to use hot glue. To attach the mirror, place it face down on a flat surface and put a drop of hot glue in the center. Carefully dip the motor shaft in the glue, keeping it at a slight angle to the perpendicular with respect to the mirror, until the glue hardens.

A better way would be to use a motor flange with a slanted face to which you can easily stick the mirror. But where would you find such a part? You can make it yourself using 3D printing! [Figure 13-10\(a\)](#) shows a 3D design I created using the free, open source program

called OpenSCAD. You can download the design from this book's GitHub repository. **Figure 13-10(b)** shows the 3D printed parts. The mirror the laser strikes first will use the flange with the lesser tilt (5 degrees), and the second mirror will use the one with more tilt (10 degrees).



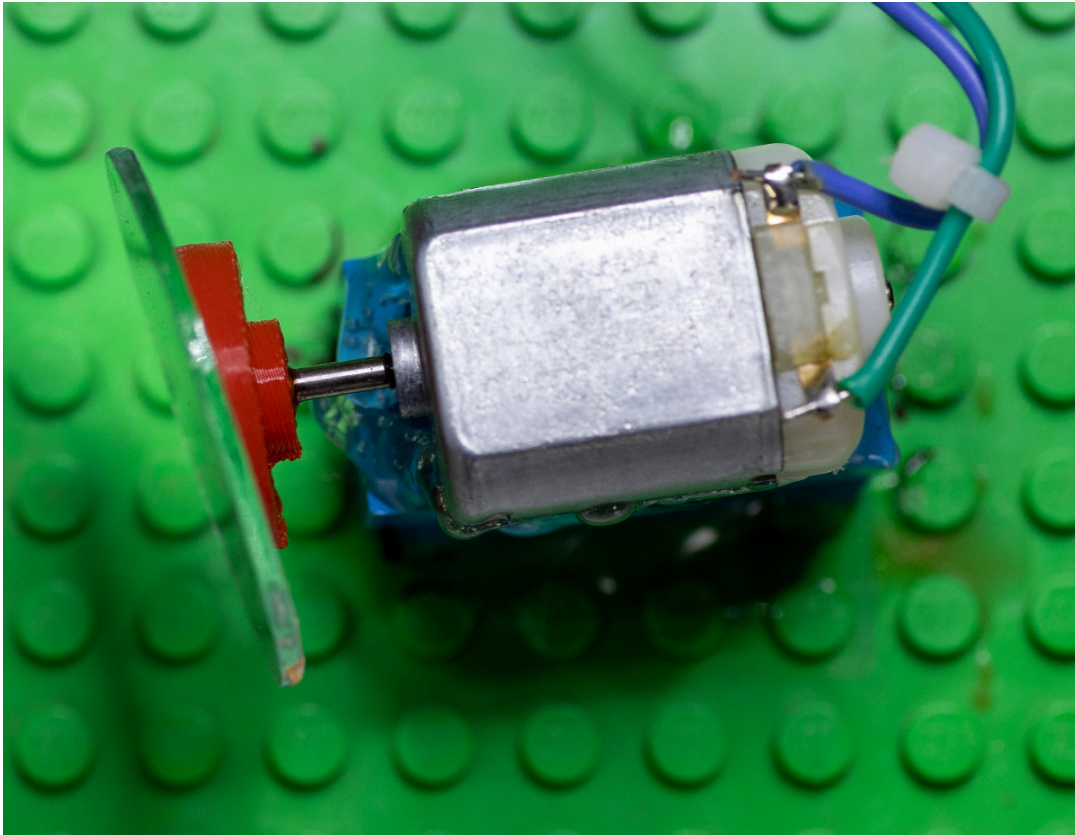
(a)



(b)

Figure 13-10: An OpenSCAD model (a) and 3D-printed flanges (b)

Print the flanges yourself if you have a 3D printer, or get them printed from a 3D printing service. (Either way, it will be inexpensive.) Once you have the parts, use superglue to attach the flanges to the motor shafts and the mirrors to the flanges. **Figure 13-11** shows the fully assembled parts.



***Figure 13-11:** Attach the mirrors to each motor shaft at a slight angle.*

To test the assembly, spin the mirror with your hand while shining the laser module at it. You should find the reflection of the laser dot moves in an ellipse when projected on a flat surface. Do the same for the second mirror. It should create a wider ellipse because of the larger angle with respect to the motor shaft.

Aligning the Mirrors

Next, align the laser module with the mirrors so that the laser reflects from mirror A to mirror B, as shown in **Figure 13-12**. Be sure that the reflected laser light from mirror A stays within the circumference of

mirror B for mirror A's entire range of rotation. (This will take some trial and error.) To test the arrangement, manually rotate mirror A. Also, be sure to position mirror B so that the light reflected from its surface will fall on a flat surface (like a wall) for the full range of rotation of both mirrors.

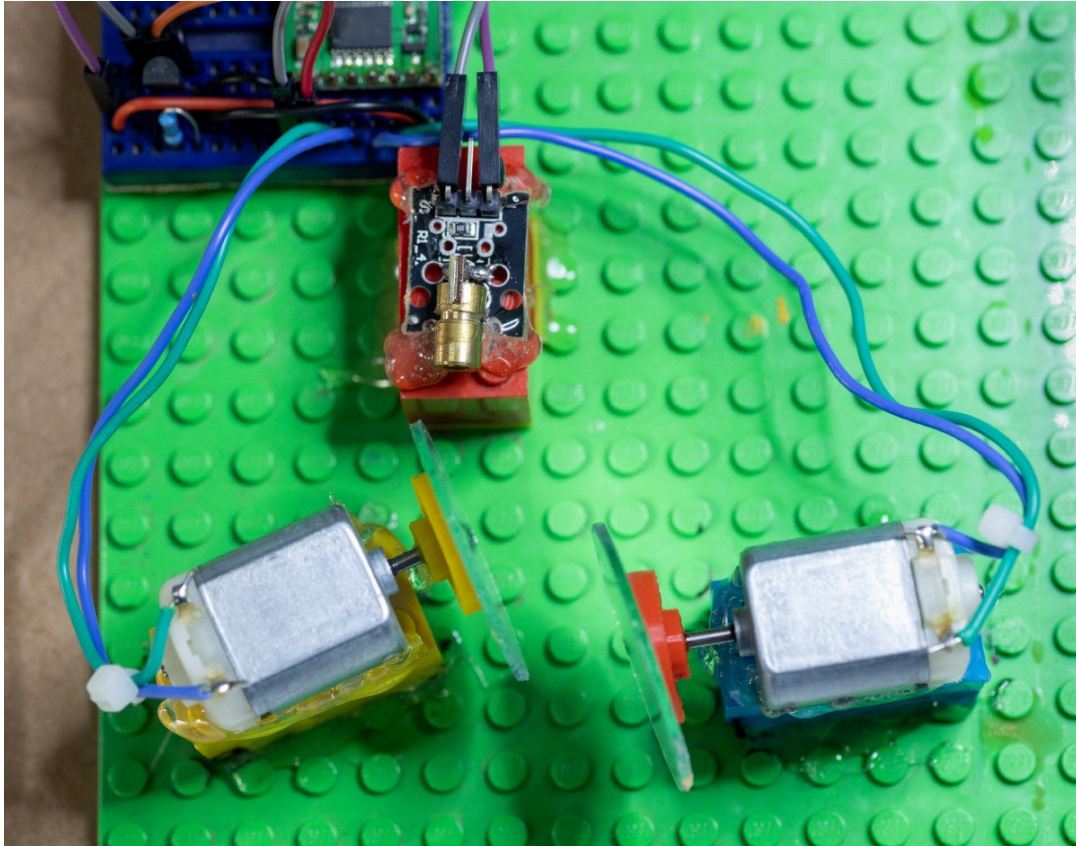


Figure 13-12: The alignment of the laser and the mirrors

NOTE As you tweak the alignment, you'll need to keep the laser pointer on. You can do this by running the project code as follows: `python Laser_audio.py --test_laser`. This command simply turns on the MOSFET controlling the laser module, as we'll discuss later in the chapter.

Once you're happy with the placement of the mirrors, hot-glue the laser module and the two motors with attached mirrors onto three identical blocks (LEGO blocks work great!) to raise them up so that the motors will be able to rotate freely. Next, place the blocks on the

mounting board, and when you’re happy with their arrangement, mark the location of each by tracing their edge with a pencil. Then hot-glue the blocks onto the board. Or use a LEGO baseplate and simply attach the LEGO blocks to the baseplate.

Powering the Motors

If your motors didn’t come with wires attached to their terminals (most don’t), solder wires to both terminals, being sure to leave sufficient wire (say, 6 inches) so that you can attach the motors to the motor driver board. The motors can be powered by a 3.7 V lithium-ion battery, or a 4x AA battery pack.

Hooking Up the Hardware

Now to hook up the hardware. You need to connect the Raspberry Pi, the motor driver board, the MOSFET, the laser module board, and the motors. The Raspberry Pi has a collection of GPIO pins for connecting to other hardware. To understand the pin layout, I highly recommend you visit the website <https://pinout.xyz>. It gives you a handy visual reference and explains the various pin functions.

NOTE *There are a few different conventions for referring to pin numbers on a Raspberry Pi. For this project, we’ll use the BCM pin-numbering convention.*

Table 13-1 lists the connections you need to make.

Table 13-1: Hardware Wiring Connections

From	To
Raspberry Pi GPIO 12	TB6612FNG PWMA
Raspberry Pi GPIO 13	TB6612FNG PWMB

From	To
Raspberry Pi GPIO 7	TB6612FNG AIN1
Raspberry Pi GPIO 8	TB6612FNG AIN2
Raspberry Pi GPIO 5	TB6612FNG BIN1
Raspberry Pi GPIO 6	TB6612FNG BIN2
Raspberry Pi GPIO 22	TB6612FNG STBY
Raspberry Pi GND	TB6612FNG GND
Raspberry Pi 3V3	TB6612FNG VCC
Raspberry Pi GPIO 25	BS170 GATE (also to GND via 10 kΩ resistor)
Raspberry Pi GND	BS170 SOURCE
Laser module GND	BS170 DRAIN
Laser module VCC	Battery pack VCC (+)
Battery pack GND (-)	TB6612FNG GND
Battery pack VCC (+)	TB6612FNG VM
Motor #1 connector #1 (polarity doesn't matter)	TB6612FNG A01
Motor #1 connector #2 (polarity doesn't matter)	TB6612FNG A02

From	To
Motor #2 connector #1 (polarity doesn't matter)	TB6612FNG B01
Motor #2 connector #2 (polarity doesn't matter)	TB6612FNG B02
Raspberry Pi 3.5 mm audio jack	AUX input of powered speaker

Figure 13-13 shows everything wired up.

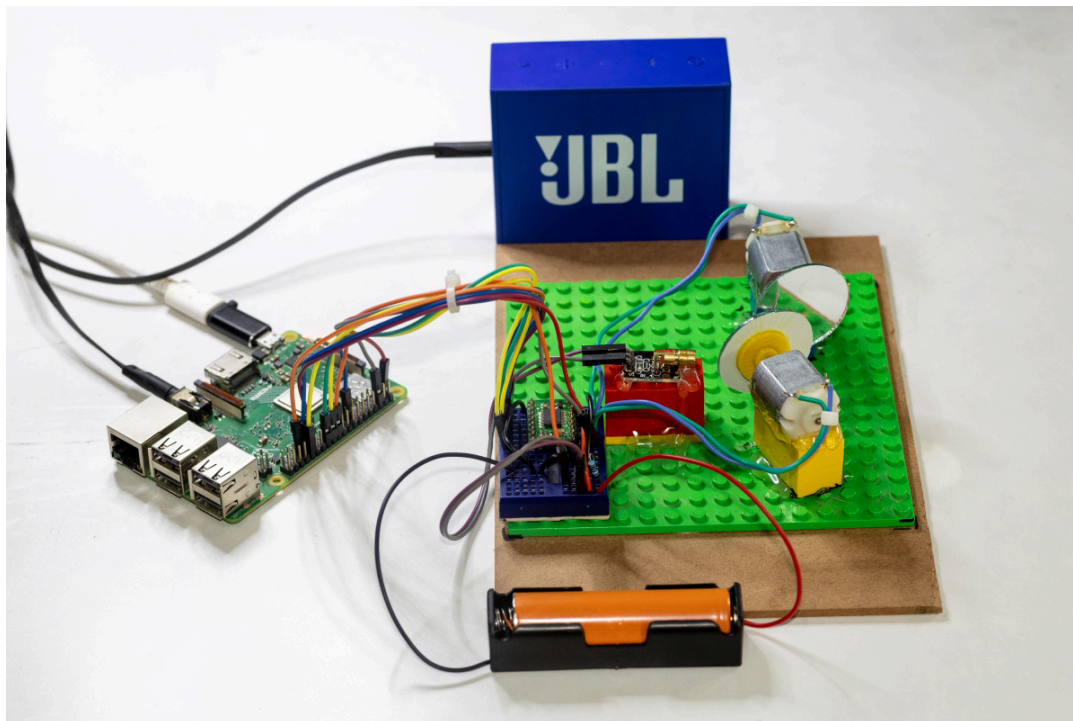


Figure 13-13: The completely wired-up laser display

Now let's look at the code.

The Code

The code for this project is in the file *laser_audio.py*. You'll begin with some basic setup. Then you'll define functions to operate and test the motors and laser, as well as a function to process audio data from a WAV file and control the motors based on that data. You'll bring everything together and accept command line options through the `main()` function. To see the full program, skip ahead to **[“The Complete Code”](#)** on **[page 305](#)**. You can also download the code at **https://github.com/mkvenkit/pp2e/tree/main/laser_audio**.

Setting Up

Start by importing the required modules:

```
import RPi.GPIO as GPIO
```

```
import time
```

```
import argparse
```

```
import pyaudio
```

```
import wave
```

```
import numpy as np
```

The `RPi.GPIO` module lets you use the pins of the Raspberry Pi. You'll use the `time` module to put delays in your code, and you'll use `argparse` to add command line arguments to the program. The `pyaudio` and `wave` modules will help you read data from WAV files and output an audio stream. Finally, you'll use `numpy` to compute the FFT of the audio data.

Next, you initialize a few global variables:

```
# define pin numbers
```

```
# uses TB6612FNG motor driver pin naming
```

```
PWMA = 12
```

```
PWMB = 13
```

```
AIN1 = 7
```

```
AIN2 = 8
```

```
BIN1 = 5
```

```
BIN2 = 6
```

```
STBY = 22
```

```
LASER = 25
```

This code stores the pin numbers for all the Raspberry Pi pins used in the project. `PWMA`, `PWMB`, `AIN1`, `AIN2`, `BIN1`, `BIN2`, and `STBY` are pins that connect to the TB6612FNG motor driver. The `LASER` pin will connect to the gate of the MOSFET, which can turn the laser module on and off. Note that you're using the BCM pin-numbering convention here.

Continue with a few more global variables:

```
# global PWM objects
```

```
pwm_a = None
```

```
pwm_b = None
```

```
# size of audio data read in
```

```
CHUNK = 2048
```

```
# FFT size
```

```
N = CHUNK
```

Here you initialize the variables `pwm_a` and `pwm_b` that will represent PWM objects, which you'll use to control the motors. You set them to `None` here because it's too early in the code to create the actual PWM objects. You also set `CHUNK`, the number of audio data samples you'll read in from the WAV file at a time, and `N`, the number of samples you'll use to compute the FFT.

You finish setting up by initializing the GPIO pins. This is necessary to use the pins. Define a function `init_pins()` for this purpose:

```
def init_pins():
```

```
    """set up pins"""
```

```
    ❶ global pwm_a, pwm_b
```

```
    # use BCM pin numbering
```

```
    ❷ GPIO.setmode(GPIO.BCM)
```

```
    # put pins into a list
```

```
    pins = [PWMA, PWMB, AIN1, AIN2, BIN1, BIN2, STBY, LASER]
```

```
    # set up pins as outputs
```

```
    ❸ GPIO.setup(pins, GPIO.OUT)
```

```
    # set PWM
```

```
    pwm_a = GPIO.PWM(PWMA, 100)
```

```
pwm_b = GPIO.PWM(PWMB, 100)
```

First you indicate that `pwm_a` and `pwm_b` are global variables ❶, since you'll be setting them inside this function. Then you set the pin mode to the BCM numbering convention ❷. Next, you put the pin variables you set earlier into a `pins` list so you can declare them all to be output pins with one call ❸. Finally, you create two `PWM` objects and assign them to your global `pwm_a` and `pwm_b` variables. The argument `100` is the frequency, in hertz, of the signal that will drive each motor. You'll vary the duty cycles of those signals to control the motors' speeds using pulse width modulation.

Controlling the Hardware

You need some helper functions to control the laser module and the motors. Let's look first at the function that toggles the laser module:

```
def laser_on(on):
```

```
    # pin 25 controls laser ctrl mosfet
```

```
    GPIO.output(LASER, on)
```

This function takes a parameter `on` that will be a Boolean `True/False` value. You pass that parameter to the `GPIO.output()` method to set the `LASER` pin on (`True`) or off (`False`). This will trigger the MOSFET to turn the laser module on or off.

Next, define a function `start_motors()` to turn the motors on at the start of the project:

```
def start_motors():
```

```
    """start both motors"""
```

```
    # enable driver chip
```

❶ `GPIO.output(STBY, GPIO.HIGH)`

`# set motor direction for channel A`

❷ `GPIO.output(AIN1, GPIO.HIGH)`

❸ `GPIO.output(AIN2, GPIO.LOW)`

`# set motor direction for channel B`

`GPIO.output(BIN1, GPIO.HIGH)`

`GPIO.output(BIN2, GPIO.LOW)`

`# set PWM for channel A`

`duty_cycle = 10`

❹ `pwm_a.start(duty_cycle)`

`# set PWM for channel B`

`pwm_b.start(duty_cycle)`

First you set the `STBY` (standby) pin to `HIGH` ❶, which in effect turns on the motor driver. Then you set the `AIN1` and `AIN2` pins to `HIGH` ❷ and `LOW` ❸, respectively. This will cause motor A to spin in one direction. (Swapping the `HIGH/LOW` values between the two pins would cause the motor to spin in the opposite direction.) You do the same thing for motor B. Lastly, you use your `PWM` objects to set the speed of the motors ❹. You set the duty cycle (which correlates to the motor speed) to 10 percent, a relatively low value, since this is just an initializing call.

You also need a function to stop the motors from spinning at the end of the project. Here's the definition:


```
def stop_motors():

    """stop both motors"""

    # stop PWM

    ❶ pwm_a.stop()

    ❷ pwm_b.stop()

    # brake A

    GPIO.output(AIN1, GPIO.HIGH)

    GPIO.output(AIN2, GPIO.HIGH)

    # brake B

    GPIO.output(BIN1, GPIO.HIGH)

    GPIO.output(BIN2, GPIO.HIGH)

    # disable driver chip

    ❸ GPIO.output(STBY, GPIO.LOW)
```

To stop the motors from spinning, you first stop the PWM signals going to the `PWMA` ❶ and `PWMB` ❷ pins. Then you set the `AIN1`, `AIN2`, `BIN1`, and `BIN2` pins all to `HIGH`, which has the effect of “braking” each motor to a stop. Finally, you disable the motor driver by setting the `STBY` pin to `LOW` ❸. Standby mode saves power when the motors aren’t required to function.

You need one more helper function to set the speed and direction of both motors. You’ll use this function to adjust the motors based on your real-time audio analysis.

```
def set_motor_speed_dir(dca, dcb, dira, dirb):
```

```
    """set speed and direction of motors"""
```

```
    # set duty cycle
```

```
    ❶ pwm_a.ChangeDutyCycle(dca)
```

```
    pwm_b.ChangeDutyCycle(dcb)
```

```
    # set direction A
```

```
    ❷ if dira:
```

```
        GPIO.output(AIN1, GPIO.HIGH)
```

```
        GPIO.output(AIN2, GPIO.LOW)
```

```
    ❸ else:
```

```
        GPIO.output(AIN1, GPIO.LOW)
```

```
        GPIO.output(AIN2, GPIO.HIGH)
```

```
    if dirb:
```

```
        GPIO.output(BIN1, GPIO.HIGH)
```

```
        GPIO.output(BIN2, GPIO.LOW)
```

```
    else:
```

```
        GPIO.output(BIN1, GPIO.LOW)
```

```
        GPIO.output(BIN2, GPIO.HIGH)
```

The `set_motor_speed_dir()` function takes four parameters: `dca` and `dcb` determine the duty cycle for each motor, while `dira` and `dirb` are Booleans that set the motors' direction of rotation. You use the `ChangeDutyCycle()` method to update the duty cycles (speeds) of the motors to the values passed into the function ❶. Then you attend to the motors' directions. If `dira` is `True` ❷, you set pins `AIN1` and `AIN2` to `HIGH` and `LOW`, respectively, which will let motor A spin in one direction. However, if `dira` is `False` ❸, you set the pins the opposite way, which will spin the motor in the other direction. You do the same thing for motor B using the `dirb` parameter.

Processing the Audio

The heart of this project is the `process_audio()` function, which reads audio data from a WAV file, outputs an audio stream with `pyaudio`, analyzes the audio data by computing an FFT, and uses the resulting analysis to control the motors. We'll look at the function in sections.

```
def process_audio(filename):
```

```
    print("opening {}".format(filename))
```

```
    # open WAV file
```

```
    ❶ wf = wave.open(filename, 'rb')
```

```
    # print audio details
```

```
    ❷ print("SW = {}, NCh = {}, SR = {}".format(wf.getsampwidth(),
```

```
        wf.getnchannels(), wf.getframerate()))
```

```
    # check for supported format
```

```
    ❸ if wf.getsampwidth() != 2 or wf.getnchannels() != 1:
```

```
print("Only single channel 16 bit WAV files are supported!")

wf.close()

return

# create PyAudio object

❷ p = pyaudio.PyAudio()

# open an output stream

❸ stream =
p.open(format=p.get_format_from_width(wf.getsampwidth()),

        channels=wf.getnchannels(),

        rate=wf.getframerate(),

        output=True)

# read first frame
```

You start by using the `wave` module to open the audio file passed into the `process_audio()` function ❶. The `wave.open()` function returns a `Wave_read` object, which you'll use to read data from the WAV file. You print out some information about the WAV file read in ❷: `SW` is the sample width in bytes, `NCh` is the number of channels in the audio, and `SR` is the sampling rate. To keep the project simple, you'll support only single-channel, 16-bit WAV files as input. You check for these specs ❸, and if the input doesn't match, you return from the function.

Next, you create the `PyAudio` object that you'll use to stream data from the WAV file to the output ❹. Then you open a `pyaudio` output stream (as indicated by the `output=True` argument), configuring it to

have the same sample width, number of channels, and sample rate as the WAV file ❹. For the Raspberry Pi, the default sound output is the 3.5 mm audio jack on the board. As long as your speaker is plugged into this jack, you'll hear the sound output.

Here's the next part of the function:

```
❶ data = wf.readframes(CHUNK)

❷ buf = np.frombuffer(data, dtype=np.int16)

# store sample rate

❸ SR = wf.getframerate()

# start motors

start_motors()

# laser on

laser_on(True)
```

Here you read `CHUNK` samples from the WAV file into variable `data` ❶. Remember, you've set `CHUNK` to `2048`, and each sample is 2 bytes wide, so you'll be reading a total of 2,048 16-bit values. You read only one chunk of data because of how the function's main loop is structured, as you'll see soon.

The `readframes()` method returns a `bytes` object, but you use the `numpy` library's `frombuffer()` function to convert the `bytes` object into a `numpy` array of 16-bit integers called `buf` ❷. You store the sampling rate (the `wave` module calls it the *frame rate*) in variable `SR` ❸; you'll need it later. Then you call the `start_motors()` and `laser_on()` functions, which we've already discussed, to get the motors and the laser module going.

Next, you enter the function's main loop, which outputs the audio and performs the FFT. The loop works on one chunk of audio data at a time, which is why you read only a single chunk of data in the previous code listing. Notice that the loop occurs inside a `try` block. Later, you'll write an `except` block to handle any problems that arise during the execution of the loop.

```
# read audio data from WAV file

try:

    # loop till there is no data to be read

    ❶ while len(data) > 0:

        # write stream to output

        ❷ stream.write(data)

        # ensure enough samples for FFT

        ❸ if len(buf) == N:

            ❹ buf = np.frombuffer(data, dtype=np.int16)

            # do FFT

            ❺ fft = np.fft.rfft(buf)

            ❻ fft = np.abs(fft) * 2.0/N

            # calc levels

            # get average of 3 frequency bands

            # 0-100 Hz, 100-1000 Hz, and 1000-2500 Hz
```



```

⑦ levels = [np.sum(fft[0:100])/100,

             np.sum(fft[100:1000])/900,

             np.sum(fft[1000:2500])/1500]

```

The main loop repeats until `data` is empty ❶, meaning you've gotten to the end of the WAV file. In the loop, you write the current chunk of data to the `pyaudio` output stream ❷. This will allow you to hear the WAV file you're processing at the same time that you're driving the motors. Then you check whether you have `N` samples in the current chunk of data to calculate the FFT ❸ (you set `N` to `2048`, the same as the chunk size, at the start of the code). This check is required because the last chunk of data read in may not have the full number of samples needed for the FFT. In that case, you'll simply skip calculating the FFT and updating the motors, since the audio file is basically over anyway.

Next, you load the audio data into a `numpy` array of 16-bit integers ❹. With the data in this format, computing the FFT is quite straightforward: you simply use the `rfft()` method from the `numpy.fft` module ❺. This method takes a signal composed of *real numbers* (like the audio data) and computes the FFT, which generally results in a set of *complex numbers*. You want to keep working with real numbers, however, so you use the `abs()` method to get the magnitudes of these complex numbers, which are real ❻. The `2.0/N` is a normalization factor you use to map the FFT values to the expected range.

Continuing the loop, you extract the relevant information from the FFT to control the motors. To analyze the audio signal, you split the frequency range into three bands: 0 to 100 Hz (bass), 100 to 1,000 Hz (midrange), and 1,000 to 2,500 Hz (treble). You're especially interested in the bass and midrange frequency bands, which roughly correspond to the beat and the vocals in a song, respectively. You compute the average amplitude value of the frequencies in each band by using the

`numpy.sum()` method and dividing the result by the number of frequencies in that band ⑦. You store the three averages in a Python list.

Notice that you're doing two different things in the `while` loop: sending audio to the output and computing the FFT of the same audio. You're able to do this and maintain the audio output because the `numpy` FFT computation is fast enough—it finishes before the current chunk of audio data has finished playing. Try an experiment: put a time delay right after FFT, and see what happens to the sound output!

Now you need to convert the average amplitudes from the FFT into motor speeds and directions, still within the `while` loop started in the previous listing. The speeds need to be percentages, while the directions need to be `True/False` values.

```
# speed1
```

```
① dca = int(5*levels[0]) percent 60
```

```
# speed2
```

```
② dcb = int(100 + levels[1]) percent 60
```

```
# dir
```

```
dira = False
```

```
dirb = True
```

```
③ if levels[2] > 0.1:
```

```
    dira = True
```

```
# set motor direction and speed
```

④ `set_motor_speed_dir(dca, dcb, dira, dirb)`

First you take the value from the lowest frequency band, scale it by a factor of `5`, convert it to an integer, and use the modulus operator (`percent`) to ensure that the value lies within the `[0, 60]` range ❶. This value controls the speed of motor A. Then you add `100` to the value from the middle frequency band and again use the modulus operator to place it in the `[0, 60]` range ❷. This value controls the speed of motor B.

NOTE *It's not a good idea to run the motors too fast at first, which is why this code limits the motors' speeds to 60 percent. Once you're confident the display is working, you can try increasing the speed threshold at ❶ and ❷.*

By default, you set `dira` to `False` to run motor A in one direction, but you switch to the other direction if the value from the highest frequency band crosses above a threshold of `0.1` ❸. Meanwhile, you keep motor B's direction constant by setting `dirb` to `True`. Finally, you call your `set_motor_speed_dir()` function to run the motors at the speeds and directions you've calculated ❹.

NOTE *There's no particularly elegant rule governing how you convert the FFT information to motor speeds and directions. The FFT values change constantly with the audio signal, so any method you come up with will change the laser pattern in response to the music. I arrived at the method described here through trial and error; I looked at FFT values while playing various types of music and chose calculations that produced a nice variety of patterns. I encourage you to play with the calculations and create your own conversions. There are no wrong answers here, as long as your method puts the motor speeds in the `[0, 100]` range (or less to avoid high speeds) and sets the directions to be `True` or `False`.*

Here's the remainder of the `process_audio()` function:

```
# read next
```

```
❶ data = wf.readframes(CHUNK)
```

```
❷ except BaseException as err:
```

```
    print("Unexpected {}, type={}".format(err, type(err)))
```

```
❸ finally:
```

```
    print("Finally: Pyaudio clean up...")
```

```
    stream.stop_stream()
```

```
    stream.close()
```

```
    # stop motors
```

```
    stop_motors()
```

First you conclude the `while` loop by reading in the next chunk of audio data to be processed ❶. Recall that the `while` loop unfolds inside a `try` block. The `except` block ❷ catches any exceptions that may arise during the loop. For example, pressing CTRL-C while the program is running will throw an exception and halt the loop, as will any errors reading the data. You end by doing some cleanup in a `finally` block ❸ that will execute whether or not any exceptions are thrown. In this block, you stop the `pyaudio` output stream and close it, and you call your `stop_motors()` function to stop the motors from spinning.

Testing the Motors

For testing purposes, it would be useful to be able to manually set the speed and direction of the motors and view the resulting laser pattern. Here's a `test_motors()` function that makes this possible:

```
def test_motors():

    """test motors by manually setting speed and direction"""

    # turn laser on

    ❶ laser_on(True)

    # start motors

    ❷ start_motors()

    # read user input

    try:

        while True:

            ❸ print("Enter dca dcb dira dirb (eg. 50 100 1 0):")

            # read input

            str_in = input()

            # parse values

            ❹ vals = [int(val) for val in str_in.split()]

            # sanity check

            if len(vals) == 4:

                ❺ set_motor_speed_dir(vals[0], vals[1], vals[2], vals[3])

            else:

                print("Input error!")
```

except:

```
print("Exiting motor test!")
```

⑥ finally:

```
# stop motors
```

```
stop_motors()
```

```
# turn laser off
```

```
laser_on(False)
```

You begin by turning on the laser ① and starting the motors ②. Then you enter a loop to get information from the user. The loop prompts the user to enter four integer values ③: `dca` and `dcB` are the duty cycles (speeds) for the motors (from 0 to 100), and `dira` and `dirb` are the motor directions (0 or 1). You wait for the input, and then parse it, using `split()` to divide the input string into separate strings based on whitespace and a list comprehension to convert each substring into an integer ④. After a sanity check to ensure that you did, in fact, get four numbers as input, you run the motors using the provided values ⑤.

This runs in a loop, so you can try inputting various speed and direction values to see the result. Since the `while` loop is placed inside a `try` block, pressing CTRL-C will throw an exception and exit from the test when you're ready. Then, in the `finally` block ⑥, you stop the motors and turn the laser off.

Putting It All Together

As usual, the `main()` function takes in command line arguments and sets the project in motion. Let's look at the command line arguments first:

```
def main():

    """main calling function"""

    # set up args parser

    ❶ parser = argparse.ArgumentParser(description="A laser audio
display.")

    # add arguments

    parser.add_argument('--test_laser', action='store_true',
required=False)

    parser.add_argument('--test_motors', action='store_true',
required=False)

    parser.add_argument('--wav_file', dest='wav_file', required=False)

    args = parser.parse_args()
```

Here you follow the familiar pattern of creating an `ArgumentParser` object to parse command line arguments for the program ❶. The program will support three different command line arguments. The `--test_laser` option just turns the laser on and is useful when you're building the motor and laser assembly. The `--test_motors` option is for testing the motors, and the `--wav_file` option lets you specify the WAV file to be read in for the laser audio display.

Here's the rest of the `main()` function:

```
# initialize pins
```

```
❶ init_pins()
```

```
# main loop
```


try:

❷ if args.test_laser:

print("laser on...")

laser_on(True)

try:

wait in a loop

while True:

time.sleep(0.1)

except:

turn laser off

laser_on(False)

❸ elif args.test_motors:

print("testing motors...")

test_motors()

❹ elif args.wav_file:

print("starting laser audio display...")

process_audio(args.wav_file)

except (Exception) as e:

print("Exception: {}".format(e))

```
print("Exiting.")

# turn laser off

❶ laser_on(False)

# call at the end

❷ GPIO.cleanup()

print("Done.")
```

You call the `init_pins()` function you defined earlier to initialize the Raspberry Pi's GPIO pins ❶. Next, you go through and process the command line arguments. If the user typed in the `--test_laser` argument, `args.test_laser` will be set to `True`. You handle this case by turning on the laser and waiting for the user to press CTRL-C to terminate the loop ❷. Similarly, you handle the `--test_motors` option by calling `test_motors()` ❸. To start the laser audio display, the user needs to use the `--wav_file` command line argument. In this case ❹, you call the `process_audio()` function. Once again, all this is embedded in a `try` block, so when the user presses CTRL-C, you break out of the loop for any of the three modes. To finish, you turn the laser off ❺ and do a GPIO cleanup ❻ before exiting the program.

Running the Laser Display

To test the project, assemble the hardware, make sure the battery pack is connected, and position everything so the laser is projecting onto a flat surface such as a wall. Then use SSH to log in to your Raspberry Pi as discussed in [Appendix B](#) and run the program from a shell. I recommend testing the laser display part first by running the program in test mode.

WARNING *The project has high-speed spinning mirrors. Please wear appropriate eye protection or cover the setup with a transparent box before running the program to avoid injuries.*

Here's a sample run of the test mode:

```
$ python laser_audio.py --test_motors
```

testing motors...

Enter dca dcb dira dirb (eg. 50 100 1 0):

```
30 40 0 1
```

Enter dca dcb dira dirb (eg. 50 100 1 0):

```
40 30 1 0
```

You can use this test to run both motors through various combinations of speeds and directions. You should see different laser patterns projected onto your wall as you change the values. To stop the program and the motors, press CTRL-C. Note that if you enter duty cycle (speed) values greater than 80, the motors will spin really fast. Be careful!

If the test succeeds, you're ready to move on to the real show. Copy a WAV file of your favorite music to the Raspberry Pi. Remember that to keep things simple, the program accepts only single-channel WAV files in 16-bit format. You can convert any audio file to this format using the free software Audacity. (A sample file is also available in the project's GitHub repository.) When the audio file is in place, run the program as follows, substituting your desired filename after the `--wave_file` option:

```
python3 laser_audio.py --wav_file bensound-allthat-16.wav
```

You should see the laser display produce lots of interesting patterns that change in time with the music, as shown in **Figure 13-14**.

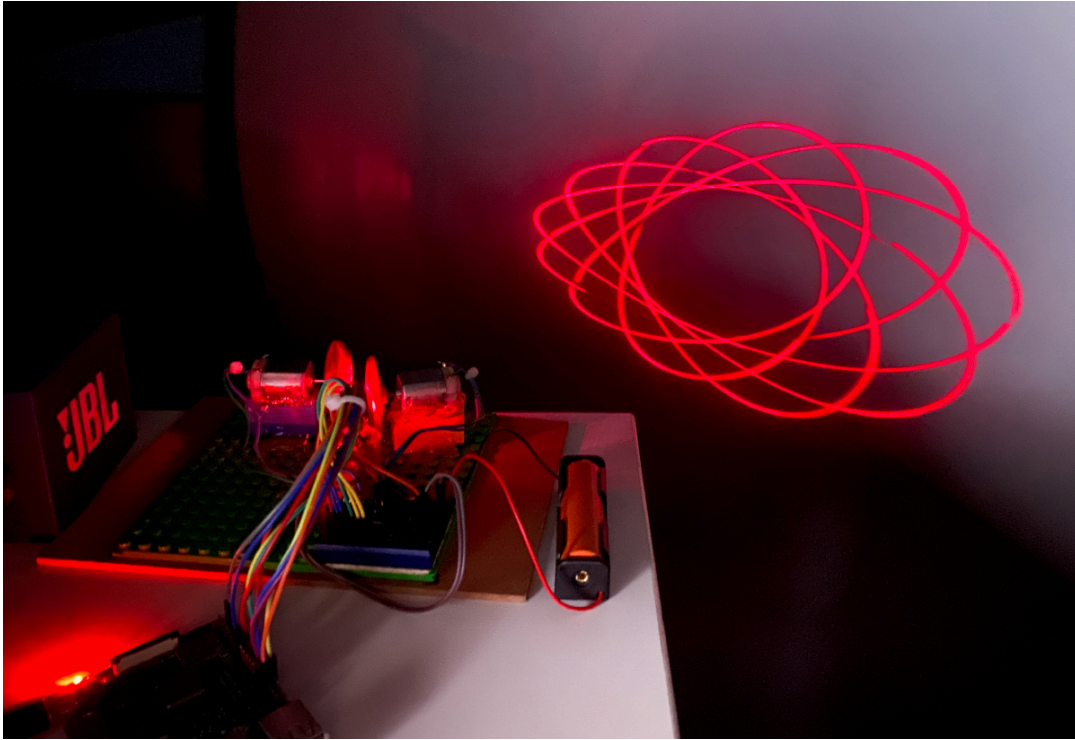


Figure 13-14: The complete wiring of the laser display and a pattern projected on the wall

Try experimenting with different WAV files, or with different calculations for converting the FFT information to motor settings, to see how the visualization responds.

Summary

In this chapter, you upped your Python and hardware skills by building a fairly complex project. You learned how to control motors with Python, a Raspberry Pi, and a motor driver. You used `numpy` to compute the FFT of audio data, and you used `pyaudio` to stream audio output in real time. You even learned to control a laser using a MOSFET!

Experiments!

Here are some ways you can modify this project:

1. The program used an arbitrary scheme to convert the FFT values into motor speed and direction data. Try changing this scheme. For example, experiment with different frequency bands and criteria for changing motor directions.
2. In this project, you converted frequency information gathered from the audio signal into motor speed and direction settings. Try making the motors move according to the overall “pulse” or volume of the music instead. For this, you can compute the *root mean square (RMS)* value of the amplitude of the signal. This computation is similar to the FFT calculation. Once you read in a chunk of audio data and put it into a `numpy` array `x`, you can compute the RMS value as follows:

```
rms = np.sqrt(np.mean(x**2))
```

Also, remember that the amplitude in your project was expressed as a 16-bit signed integer, which can have a maximum value of 32,768 (a useful number to keep in mind for normalization). Use this RMS amplitude in conjunction with the FFT to generate a greater variation of laser patterns.

3. You know now that the frequency content, and hence the FFT of the audio data, changes in sync with the audio. Can you create a real-time visualization like the one shown in **Figure 13-15**, of both the audio data and the FFT as the audio plays through a speaker? This is intended to run on your computer, not your Raspberry Pi.

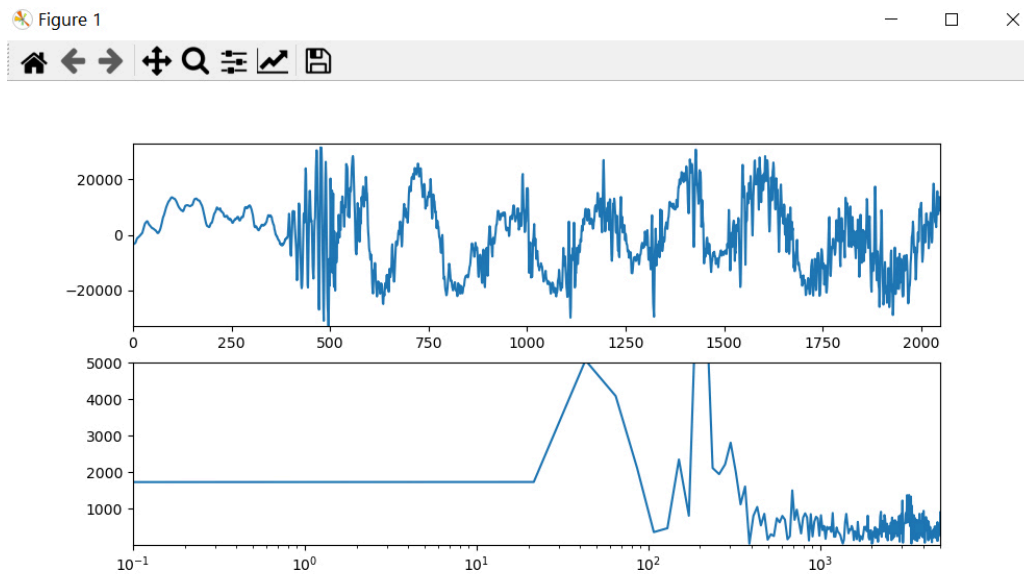


Figure 13-15: A real-time FFT visualization

Here are a few hints to solve this problem:

1. Use `matplotlib` for plotting.
2. Use Python's `multiprocessing` package so your music streaming output and plotting can happen simultaneously.
3. Use the `numpy.fft.rfftfreq()` method to get the frequencies corresponding to the FFT values, for ease of plotting.

(The solution code for this experiment is in the book's GitHub repository, but give it a shot yourself first!)

The Complete Code

Here's the complete Python code for this project:

```
"""
```

```
laser_audio.py
```

```
Creates a laser display that changes in time to music.
```

```
Uses Python on a Raspberry Pi.
```

Author: Mahesh Venkitachalam

"""

import RPi.GPIO as GPIO

import time

import argparse

import pyaudio

import wave

import numpy as np

define pin numbers

uses TB6612FNG motor driver pin naming

PWMA = 12

PWMB = 13

AIN1 = 7

AIN2 = 8

BIN1 = 5

BIN2 = 6

STBY = 22

LASER = 25

global PWM objects


```
pwm_a = None

pwm_b = None

# size of audio data read in

CHUNK = 2048

# FFT size

N = CHUNK

def init_pins():

    """set up pins"""

    global pwm_a, pwm_b

    # use BCM pin numbering

    GPIO.setmode(GPIO.BCM)

    # put pins into a list

    pins = [PWMA, PWMB, AIN1, AIN2, BIN1, BIN2, STBY, LASER]

    # set up pins as outputs

    GPIO.setup(pins, GPIO.OUT)

    # set PWM

    pwm_a = GPIO.PWM(PWMA, 100)

    pwm_b = GPIO.PWM(PWMB, 100)

def laser_on(on):
```

```
"""turn laser MOSFET on/off"""

# pin 25 controls laser ctrl mosfet

GPIO.output(LASER, on)

def test_motors():

    """test motors by manually setting speed and direction"""

    # turn laser on

    laser_on(True)

    # start motors

    start_motors()

    # read user input

    try:

        while True:

            print("Enter dca dcb dira dirb (eg. 50 100 1 0):")

            # read input

            str_in = input()

            # parse values

            vals = [int(val) for val in str_in.split()]

            # sanity check

            if len(vals) == 4:
```

```
        set_motor_speed_dir(vals[0], vals[1], vals[2], vals[3])

    else:

        print("Input error!")

except:

    print("Exiting motor test!")

finally:

    # stop motors

    stop_motors()

    # turn laser off

    laser_on(False)

def start_motors():

    """start both motors"""

    # enable driver chip

    GPIO.output(STBY, GPIO.HIGH)

    # set motor direction for channel A

    GPIO.output(AIN1, GPIO.HIGH)

    GPIO.output(AIN2, GPIO.LOW)

    # set motor direction for channel B

    GPIO.output(BIN1, GPIO.HIGH)
```

```
GPIO.output(BIN2, GPIO.LOW)

# set PWM for channel A

duty_cycle = 0

pwm_a.start(duty_cycle)

# set PWM for channel B

pwm_b.start(duty_cycle)

def stop_motors():

    """stop both motors"""

    # stop PWM

    pwm_a.stop()

    pwm_b.stop()

    # brake A

    GPIO.output(AIN1, GPIO.HIGH)

    GPIO.output(AIN2, GPIO.HIGH)

    # brake B

    GPIO.output(BIN1, GPIO.HIGH)

    GPIO.output(BIN2, GPIO.HIGH)

    # disable driver chip

    GPIO.output(STBY, GPIO.LOW)
```

```
def set_motor_speed_dir(dca, dcb, dira, dirb):
```

```
    """set speed and direction of motors"""
```

```
    # set duty cycle
```

```
    pwm_a.ChangeDutyCycle(dca)
```

```
    pwm_b.ChangeDutyCycle(dcb)
```

```
    # set direction A
```

```
    if dira:
```

```
        GPIO.output(AIN1, GPIO.HIGH)
```

```
        GPIO.output(AIN2, GPIO.LOW)
```

```
    else:
```

```
        GPIO.output(AIN1, GPIO.LOW)
```

```
        GPIO.output(AIN2, GPIO.HIGH)
```

```
    if dirb:
```

```
        GPIO.output(BIN1, GPIO.HIGH)
```

```
        GPIO.output(BIN2, GPIO.LOW)
```

```
    else:
```

```
        GPIO.output(BIN1, GPIO.LOW)
```

```
        GPIO.output(BIN2, GPIO.HIGH)
```

```
def process_audio(filename):
```

```
"""reads WAV file, does FFT and controls motors"""

print("opening {}".format(filename))

# open WAV file

wf = wave.open(filename, 'rb')

# print audio details

print("SW = {}, NCh = {}, SR = {}".format(wf.getsampwidth(),

    wf.getnchannels(), wf.getframerate()))

# check for supported format

if wf.getsampwidth() != 2 or wf.getnchannels() != 1:

    print("Only single channel 16 bit WAV files are supported!")

wf.close()

return

# create PyAudio object

p = pyaudio.PyAudio()

# open an output stream

stream =

p.open(format=p.get_format_from_width(wf.getsampwidth()),

    channels=wf.getnchannels(),

    rate=wf.getframerate(),
```

```
        output=True)

# read first frame

data = wf.readframes(CHUNK)

buf = np.frombuffer(data, dtype=np.int16)

# store sample rate

SR = wf.getframerate()

# start motors

start_motors()

# laser on

laser_on(True)

# read audio data from WAV file

try:

    # loop till there is no data to be read

    while len(data) > 0:

        # write stream to output

        stream.write(data)

        # ensure enough samples for FFT

        if len(buf) == N:

            buf = np.frombuffer(data, dtype=np.int16)
```



```
# do FFT

fft = np.fft.rfft(buf)

fft = np.abs(fft) * 2.0/N

# calc levels

# get average of 3 frequency bands

# 0-100 Hz, 100-1000 Hz, and 1000-2500 Hz

levels = [np.sum(fft[0:100])/100,

          np.sum(fft[100:1000])/900,

          np.sum(fft[1000:2500])/1500]

# speed1

dca = int(5*levels[0]) percent 60

# speed2

dcb = int(100 + levels[1]) percent 60

# dir

dira = False

dirb = True

if levels[2] > 0.1:

    dira = True

# set motor direction and speed
```

```
    set_motor_speed_dir(dca, dcb, dira, dirb)

    # read next

    data = wf.readframes(CHUNK)

except BaseException as err:

    print("Unexpected {}, type={}".format(err, type(err)))

finally:

    print("Finally: Pyaudio clean up...")

    stream.stop_stream()

    stream.close()

    # stop motors

    stop_motors()

    # close WAV file

    wf.close()

def main():

    """main calling function"""

    # set up args parser

    parser = argparse.ArgumentParser(description="A laser audio
display.")

    # add arguments
```

```
parser.add_argument('--test_laser', action='store_true',
required=False)

parser.add_argument('--test_motors', action='store_true',
required=False)

parser.add_argument('--wav_file', dest='wav_file', required=False)

args = parser.parse_args()

# initialize pins

init_pins()

# main loop

try:

    if args.test_laser:

        print("laser on...")

        laser_on(True)

        try:

            # wait in a loop

            while True:

                time.sleep(1)

        except:

            # turn laser off

            laser_on(False)
```

```
elif args.test_motors:

    print("testing motors...")

    test_motors()

elif args.wav_file:

    print("starting laser audio display...")

    process_audio(args.wav_file)

except (Exception) as e:

    print("Exception: {}".format(e))

    print("Exiting.")

# turn laser off

laser_on(False)

# call at the end

GPIO.cleanup()

print("Done.")

# call main

if __name__ == '__main__':

    main()
```
